Every stream call builds a pipeline and drops it

Bytes and objects per call, match at element 500, compact headers on.

Operation	JDK stream	Objects	EC, every version
`anyMatch` / `anySatisfy`	176 B	7	0 B
`filter().findFirst()` / `detect`	216 B	8	0 B
`filter().count()` / `count`	224 B	8	0 B

- The stream’s own cost is per call: the same at element 1 as at 1000.
- EC versions: capturing or fat-free, eager or lazy, all 0 B.

Where the bytes come from

`list.stream().filter(p).findFirst()`, one call, match at element 500.

Origin	Objects	Bytes
Stream source: `ReferencePipeline$Head`, spliterator	2	80 B
`filter` stage and its sink	2	72 B
terminal op: `FindSink`, `Optional`	2	32 B
caller lambda `Integer.valueOf(test)::equals`	1	16 B
`Integer` box, outside the cache	1	16 B
Total per call	**8**	**216 B**

Garbage you cannot read from the code

The same line of code leaves different garbage under different conditions.

- `Integer.valueOf(1)` is cached; `Integer.valueOf(500)` costs 16 B per call.
- Escape analysis removes every EC object, and none of the stream's.
- Compact headers save 16 to 24 B per call; the objects stay.
- `anyMatch(p)` is 40 B cheaper than `filter(p).findFirst()`.

Escape analysis is not a promise

With `-XX:-DoEscapeAnalysis` each lambda shape shows what it creates.

`anyMatch`, match at element 500	**EA on**	**EA off**
JDK `list.stream().anyMatch(…)`	176 B	176 B
EC lazy `asLazy().anySatisfy(…)`	0 B	48 B
EC capturing `Iterate.anySatisfy(…)`	0 B	32 B
EC fat-free `Iterate.anySatisfyWith(…)`	0 B	16 B

- The JIT removes allocations only in code it inlines.
- Turning it off is an upper bound, not a typical JVM.

From bytes per call to spikes

Bytes per call are measured; rates and spikes are derived.

Element 500	B per call	MB/s at 10 M calls/s	10 s spike
JDK `anyMatch`	176	1,760	17.6 GB
JDK `findFirst`	216	2,160	21.6 GB
JDK `count`	224	2,240	22.4 GB
EC, every version	0	0	0

Fat-free lambdas leave less behind

```
// capturing: a new lambda on every call, holding the value
Iterate.anySatisfy(list, Integer.valueOf(test)::equals);

// fat-free: one constant lambda, the value is a parameter
Iterate.anySatisfyWith(list, Integer::equals, test);

// JDK stream: the pipeline is built whatever the lambda
list.stream().anyMatch(Integer.valueOf(test)::equals);
```

- Escape analysis off: 32 B capturing, 16 B fat-free, 176 B stream.
- Less garbage per call keeps the headroom free for the next spike.

References

- Package `java.util.stream` (Java SE 25 API)
- The Java Tutorials: Lambda Expressions
- Java HotSpot VM Performance Enhancements: Escape Analysis (Java SE 25)
- Java Magazine: Escape analysis in the HotSpot JIT compiler
- HotSpot Virtual Machine Garbage Collection Tuning Guide (Java SE 25)